

LUH Cluster Example Project: Vehicle Routing Problem (VRP)

This example project demonstrates how to work with the **LUIS cluster system** at Leibniz University Hannover. It uses a Vehicle Routing Problem (VRP) optimization model with Gurobi as a practical example.

Learning Objectives

After working through this example, you will be able to:

1. **Connect to the cluster** via SSH and navigate the file system
 2. **Understand the module system** and load required software (Python, uv, Gurobi)
 3. **Manage Python environments** using uv for dependency management
 4. **Write SLURM job scripts** to run computations on compute nodes
 5. **Submit and monitor jobs** using SLURM commands ([sbatch](#), [squeue](#), [scancel](#))
 6. **Understand batch processing** by submitting multiple jobs with different parameters
 7. **Interpret output files** including logs, solutions, and error messages
-

Before You Begin

New to the LUIS cluster? Start here:

1. Read the [Quick Start Guide](#) to understand the cluster basics
2. Learn [How to Connect](#) to the cluster
3. Familiarize yourself with [Code-Server \(VS Code\)](#) - our recommended development environment

Need help? Check the [FAQ](#) or learn [How to Get Support](#).

Full documentation: [LUIS Cluster Documentation](#)

Project Structure

```
example_project/
├── main.py           # Main entry point for the VRP solver
├── build_vrp.py      # Gurobi model definition and solution
writer
├── instance_module_vrp.py  # VRP instance data class and generator
├── pyproject.toml     # Python project configuration (uv/pip)
├── cluster_files/
│   ├── job.sh         # SLURM job script (runs on compute node)
│   ├── queue.sh       # Queue script (submits multiple jobs)
│   ├── instances.txt  # List of instances to solve
│   └── cluster_commands.txt # Quick reference for cluster commands
└── vrp_instances/    # Generated VRP instance files (.vrp)
```

```
|─ vrp_solutions/      # Output solution files
|─ logs/               # SLURM job output and error logs
```

About the Vehicle Routing Problem (VRP)

The **Vehicle Routing Problem (VRP)** is a classic optimization problem in operations research:

Problem: Given a depot, a set of customers with demands, and a fleet of vehicles with capacity constraints, find the optimal set of routes that:

- Start and end at the depot
- Visit each customer exactly once
- Respect vehicle capacity limits
- Minimize total travel distance

Example: A delivery company needs to plan routes for its trucks to deliver packages to multiple customers while minimizing fuel costs.

This project implements the VRP as a **Mixed-Integer Programming (MIP)** model using Gurobi:

Component	Description
<code>instance_module_vrp.py</code>	Generates random problem instances
<code>build_vrp.py</code>	Defines the mathematical optimization model
<code>main.py</code>	Loads instances, runs solver, outputs solutions

Quick Start (TL;DR)

For experienced users, here's the minimal workflow:

1. Open [Open OnDemand](#) in your browser
2. Go to [Interactive Apps](#) → [Code-Server](#)
3. Set working directory to `$BIGWORK/projects/example_project`, configure resources, click [Launch](#)
4. Once running, click [Connect to VS Code](#)
5. Open a terminal in VS Code and run:

```
# Load modules and install dependencies
module load GCCcore/.14.2.0 uv/0.9.22
uv sync

# Submit jobs
sbatch cluster_files/queue.sh

# Monitor
squeue -u $USER
```

Getting Started

1. Connect to the Cluster via Code-Server

We use **Code-Server** (VS Code in the browser) via Open OnDemand. This runs VS Code directly on a compute node, giving you a full development environment.

1. Open [Open OnDemand](#) in your browser
2. Log in with your university credentials
3. Navigate to [Interactive Apps](#) → [Code-Server](#)
4. Configure your session:
 - **Working Directory:** `/bigwork/<your-username>/projects/example_project`
 - **Walltime:** 2-4 hours (for development)
 - **Memory:** 4G (or more if needed)
 - **CPUs:** 1-2
5. Click [Launch](#) and wait for the job to start
6. Click [Connect to VS Code](#) to open your development environment

Note: Only one Code-Server session per user is allowed at a time.

For more details, see the [Code-Server Documentation](#).

2. Navigate to Your Project

Once in VS Code, open a terminal ([Terminal](#) → [New Terminal](#)) and navigate to your project:

```
cd $BIGWORK/projects/example_project
```

Note: `$BIGWORK` is an environment variable pointing to your high-capacity storage area on the cluster.

3. Load Required Modules

Before running any Python code, load the necessary modules:

```
module load GCCcore/.14.2.0 uv/0.9.22
```

- **GCCcore:** Required dependency for uv
- **uv:** Modern Python package manager for virtual environments

Tip: Use `module spider <package>` to find available versions and dependencies.

4. Install Dependencies

This project uses `uv` for dependency management. Install the project dependencies:

```
uv sync
```

This will create a virtual environment and install all dependencies defined in `pyproject.toml`.

Running Interactively (Testing Only)

Since Code-Server runs on a compute node, you can run quick tests directly in the VS Code terminal:

```
uv run main.py instance_name=i6_b10_1 TimeLimit=30 Threads=1
```

Tip: This is great for testing and debugging your code before submitting batch jobs.

Warning: For long-running or resource-intensive computations, submit batch jobs via SLURM instead of running them in Code-Server.

Submitting Jobs to the Cluster

SLURM is the job scheduler that manages compute resources on the cluster. For comprehensive documentation, see the [SLURM Usage Guide](#).

Step 1: Generate Instances (if needed)

Generate VRP problem instances:

```
uv run instance_module_vrp.py
```

This creates 5 instances with 6 locations and vehicle capacity of 10 in the `vrp_instances/` folder.

Step 2: Configure Instances to Solve

Edit `cluster_files/instances.txt` to list the instances you want to solve (one per line):

```
i6_b10_1  
i6_b10_2  
i6_b10_3
```

Step 3: Submit Jobs

Submit all jobs using the queue script:

```
sbatch cluster_files/queue.sh
```

This will:

1. Read each instance from `instances.txt`
2. Submit a separate SLURM job for each instance
3. Each job runs `job.sh` on a compute node

Step 4: Monitor Your Jobs

Check the status of your jobs:

```
squeue -u $USER
```

View all your running/pending jobs with more details:

```
squeue -u $USER -l
```

Cancel a specific job:

```
scancel <job_id>
```

Cancel all your jobs:

```
scancel -u $USER
```

Understanding the SLURM Scripts

`queue.sh` - The Queue Manager

This script runs on the login node and submits multiple jobs:

```
#SBATCH --job-name=vrp_queue      # Job name
#SBATCH --ntasks=1                # Number of tasks
#SBATCH --mem=1G                  # Memory for the queue script itself
#SBATCH --time=00:02:00           # Time limit (short, just for
submitting)
```

Configurable parameters (modify at the top of the script):

Parameter	Default	Description
-----------	---------	-------------

Parameter	Default	Description
walltime	1	Hours per job
memory	4G	Memory per job
Threads	1	CPU cores per job
partition	smp	Cluster partition
TimeLimit	30	Gurobi solver time limit (seconds)
SoftMemLimit	3	Gurobi soft memory limit (GB)

job.sh - The Worker Script

This script runs on each compute node:

```
#!/bin/bash -l
#SBATCH --ntasks=1

# Load required modules
module load GCCcore/.14.2.0
module load uv/0.9.22

# Run the solver
uv run main.py instance_name=$instance_name TimeLimit=$TimeLimit ...
```

Output Files

After jobs complete, find your results in:

Location	Contents
vrp_solutions/solution_<instance>.txt	Optimal tour routes
logs/<instance>.out	Standard output (job info)
logs/<instance>.err	Standard error (module loading, errors)
logs/<instance>_gurobi.log	Detailed Gurobi solver log

Customizing for Your Project

1. Modify pyproject.toml

Add your Python dependencies:

```
[project]
name = "your-project"
```

```
version = "0.1.0"
requires-python = ">=3.13"
dependencies = [
    "gurobipy>=13.0.0",
    "numpy",
    "pandas",
    # Add your packages here
]
```

Then run `uv sync` to install them.

2. Adapt the Job Script

Modify `cluster_files/job.sh` to run your code:

```
# Change the main script call
uv run your_script.py your_arguments
```

3. Adjust Resource Requirements

In `queue.sh`, modify resources based on your needs:

```
export walltime=24          # For longer jobs
export memory=16G           # For memory-intensive tasks
export Threads=4            # For parallel processing
export partition=bigmem     # For high-memory jobs (if available)
```

Useful Cluster Commands

Module Management

```
module avail                # List all available modules
module list                 # List currently loaded modules
module spider <name>        # Search for a module and its dependencies
module load <name>          # Load a module
module unload <name>        # Unload a module
module purge                # Unload all modules
```

Job Management

```
squeue -u $USER            # View your jobs
squeue -p <partition>      # View jobs in a partition
sinfo                       # View cluster partitions and status
```

```
sacct -j <job_id>      # View job accounting info
scancel <job_id>       # Cancel a job
scontrol show job <id>  # Detailed job info
```

Storage

```
echo $HOME              # Home directory (limited space, backed up)
echo $BIGWORK           # Large working directory (not backed up!)
quota                   # Check your storage quota
```

Common Issues & Solutions

"Module not found"

```
module spider <module_name> # Find correct version and dependencies
```

Job stuck in "PENDING" state

- Check `sinfo` to see partition availability
- Your resource request may be too large
- Cluster may be busy - be patient

"Out of memory" errors

- Increase `memory` in `queue.sh`
- Check your code for memory leaks
- Use `SoftMemLimit` to let Gurobi manage memory

Permission denied

```
chmod +x cluster_files/*.sh # Make scripts executable
```

Quick Reference Links

Topic	Link
Cluster Documentation	docs.cluster.uni-hannover.de
Code-Server (VS Code)	Code-Server Guide
SLURM Jobs	SLURM Usage Guide
Modules & Software	Software Guide

Topic	Link
File Systems	Storage Guide
Get Support	Support Guide
Gurobi	gurobi.com/documentation
uv (Python)	docs.astral.sh/uv

Last updated: January 2026